ADA FINANCE

The AI-Native Accounting Operating System for Kenya

"Zero-effort compliance."

COMPREHENSIVE TECHNICAL IMPLEMENTATION PLAN

Deep Agentic Architecture | Ledger Design | Cybersecurity | Backend Engineering

Module-by-Module Execution Strategy with Cost Modelling

March 2026 | CONFIDENTIAL

Version 2.0 — Expanded Agent Architecture + Verification Layers

1. DEEP AGENTIC ARCHITECTURE

Ada Finance is not a CRUD application with AI bolted on. It is a multi-agent system where every accounting function is performed by a specialised agent, verified by a dedicated verification agent, and orchestrated by a central controller. This section describes every agent, its tools, its model, and its cost profile.

1.1 Agent Design Philosophy

Every agent follows the PROPOSE → VERIFY → EXECUTE pattern. No agent acts alone. Primary agents propose actions. Verification agents check them. Only after verification passes does the Orchestrator permit execution (posting to the ledger, sending to KRA, disbursing payment). This dual-agent pattern mirrors the Maker/Checker control that auditors require.

Agent Framework: All agents are built using LangChain (Python) with LangGraph for stateful, multi-step workflows. Each agent is a LangGraph node with defined input/output schemas, tool access, and state transitions. The Orchestrator is a LangGraph supervisor that routes tasks to the correct agent chain.

Tool Access: Agents interact with the outside world through strictly defined tools (functions). An agent cannot make arbitrary API calls — it can only invoke tools registered in its toolbelt. This prevents hallucinated API calls and limits blast radius.

Memory: Short-term memory is held in Redis (conversation context, current task state). Long-term memory lives in PostgreSQL (transaction history, vendor patterns, correction history). Agents retrieve relevant history before making decisions.

1.2 The Complete Agent Registry — 22 Agents

The system comprises 22 agents organised into 5 tiers: Input Agents (capture data), Processing Agents (transform data into journal entries), Verification Agents (validate everything), Execution Agents (post to ledger, file with KRA, disburse payments), and System Agents (self-improvement, monitoring, scheduling).

TIER 1: INPUT AGENTS (Data Capture)

Agent 1 — WhatsApp Listener Agent: Receives all incoming WhatsApp messages. Performs initial intent classification: is this a transaction recording, a report request, an approval response, a question, or noise? Routes to the appropriate downstream agent.

Model: Fine-tuned Llama 3.2 3B (self-hosted on CPU). This is the smallest model that can reliably classify intents. At 3B parameters it runs on CPU with <100ms latency and zero API cost.

Tools: WhatsApp message parser, media downloader (for receipt photos), user identity resolver (maps phone number to user + org), intent classifier.

Token Cost: Zero (self-hosted). Compute cost: ~$0.002 per message on shared CPU.

Agent 2 — WhatsApp Data Entry Agent: Handles the actual data extraction from WhatsApp text messages. Converts natural language into structured transaction data. This is the agent that understands “Paid 5k for fuel via M-Pesa” and extracts {type: expense, amount: 5000, category: fuel, payment_method: mpesa}.

Model: Anthropic Claude 3.5 Haiku for simple entries (single-line transactions). Claude 3.5 Sonnet for complex entries (multi-line, foreign currency, split transactions). The router (Agent 1) determines complexity.

Why Haiku for Simple Entries: Haiku costs $0.25/MTok input and $1.25/MTok output — 80% cheaper than Sonnet. A simple expense message is ~100 tokens in, ~200 tokens out = $0.000275 per entry. At 1,000 entries/month = $0.28/month. Sonnet would cost $1.40/month for the same volume.

Tools: Chart of Accounts lookup, vendor/customer search, recent transaction history (last 10 similar), M-Pesa account resolver, currency parser (handles 'k' = thousand, 'm' = million).

Output: Structured JSON: {effective_date, description, lines: [{account_code, debit, credit}], source_type: WHATSAPP, confidence_score}.

Agent 3 — Receipt/Invoice OCR Agent: Processes photos of receipts and invoices sent via WhatsApp or uploaded to the PWA. Extracts vendor name, date, line items, subtotal, VAT amount, and total.

Model: PaddleOCR (open-source, self-hosted GPU) for text detection and recognition. Then a fine-tuned Donut model (open-source, self-hosted GPU) for structured extraction. No external API calls.

Why Not Google Vision or AWS Textract: Google Vision costs $1.50/1,000 pages. AWS Textract costs $1.50/1,000 pages for tables. PaddleOCR + Donut is $0.00 per page after the GPU instance cost ($0.50/hr shared across all OCR tasks). At 500 receipts/month, Google would cost $0.75/month vs. ~$0.05/month allocated GPU cost.

Tools: Image preprocessor (rotation, contrast, deskew), PaddleOCR text extractor, Donut structured parser, VAT amount validator (checks 16% of subtotal), KRA PIN validator.

Output: Structured JSON: {vendor_name, vendor_pin, date, items: [{description, qty, unit_price, amount}], subtotal, vat_amount, total, payment_method_detected}.

Agent 4 — Email Monitor Agent: Connects to the business email inbox (IMAP or Nylas). Polls every 60 seconds. Classifies each email and extracts attachments. This agent is the "eyes" of the system — it knows when a bank statement arrives, when a supplier sends an invoice, when KRA sends a notice.

Model: Fine-tuned Llama 3.2 3B for email classification (same as WhatsApp Listener). Runs on CPU. Classifications: BANK_STATEMENT, SUPPLIER_INVOICE, CUSTOMER_RECEIPT, KRA_NOTICE, PAYSLIP, INTERNAL, SPAM.

Tools: IMAP connector (imaplib), attachment extractor, S3 uploader, sender domain resolver, subject line parser, PDF/CSV detector.

Token Cost: Zero (self-hosted). Compute: ~$0.003 per email on shared CPU.

Agent 5 — Bank Statement Parser Agent: Receives bank statement files (CSV, PDF, Excel) from the Email Monitor or manual upload. Parses them into standardised transaction rows regardless of source bank format.

Model: Rule-based parser (no LLM needed). Each bank (Equity, KCB, NCBA, Co-op, Stanbic, ABSA, StanChart) has a dedicated parser class that maps column positions to standard fields. For PDF statements, uses Tabula-py for table extraction.

Tools: CSV parser (pandas), PDF table extractor (tabula-py), Excel reader (openpyxl), bank format detector (identifies which bank based on column headers), date normaliser, amount parser (handles parentheses for negatives).

Token Cost: Zero. Pure Python rule engine. No LLM.

Agent 6 — M-Pesa Webhook Agent: Receives real-time C2B (Customer to Business) payment notifications from Safaricom Daraja API. Creates an incoming payment record and triggers matching.

Model: None (pure webhook handler). No LLM needed.

Tools: Daraja webhook validator (checks signature), transaction deduplicator, customer lookup (by phone number), invoice matcher (by bill reference or amount).

Token Cost: Zero.

TIER 2: PROCESSING AGENTS (Ledger Entry Creation)

Agent 7 — Journal Proposal Agent (The Bookkeeper): This is the primary accounting brain. It takes structured data from any input agent and converts it into a proper double-entry journal proposal. It knows the full Chart of Accounts, understands Kenyan tax rules (VAT rates, WHT categories, exempt supplies), and applies correct accounting treatment.

Model: Claude 3.5 Sonnet. This is the one place where you cannot compromise on intelligence. The journal proposal must be correct — wrong accounting entries cascade into wrong financial statements and wrong tax filings. Sonnet's structured output capability and deep reasoning make it the right choice.

System Prompt Engineering: The system prompt includes: (1) the full Chart of Accounts, (2) Kenyan VAT rules (standard 16%, zero-rated, exempt), (3) WHT rate table by vendor category, (4) NSSF/NHIF/Housing Levy formulas, (5) the last 20 similar transactions for pattern matching, (6) strict output JSON schema. Prompt is ~2,000 tokens. Average response is ~300 tokens.

Token Cost: Input: 2,000 tokens × $3/MTok = $0.006. Output: 300 tokens × $15/MTok = $0.0045. Per entry: $0.0105. At 1,000 entries/month: $10.50/month. This is the single largest API cost in the system.

Tools: Chart of Accounts fetcher, tax rate lookup, vendor WHT category resolver, recent transaction pattern matcher, VAT calculator (handles inclusive/exclusive), period validator (is the period open?), evidence linker.

Output: Complete journal entry proposal: {effective_date, description, lines: [{account_code, account_name, debit, credit, tax_code}], tax_implications: {vat_amount, wht_amount, is_exempt}, confidence: 0.0-1.0, reasoning: string}.

Agent 8 — Recurring Transaction Agent: Detects and manages recurring transactions. After 3 occurrences of a similar transaction (same vendor, similar amount, regular interval), it creates a template and auto-proposes entries on the expected date.

Model: XGBoost classifier (self-hosted). No LLM needed. Features: vendor_id, account_code, amount_bucket, day_of_month, day_of_week, interval_days. Trained on historical correction data.

Tools: Pattern detector (statistical), template manager (CRUD on recurring_templates table), next-occurrence predictor, amount variance detector (flags if >10% deviation from template).

Token Cost: Zero. Pure ML inference <1ms.

Agent 9 — Bank Reconciliation Agent: Matches imported bank transactions to ledger entries. Uses a 5-rule cascade matching engine with decreasing confidence thresholds.

Model: Hybrid: Rule engine for high-confidence matches + fine-tuned sentence-transformers/all-MiniLM-L6-v2 (open-source, 22M params) for fuzzy description matching. Self-hosted on CPU.

Matching Rules (in order): (1) Exact amount + exact date + reference number match = 99% confidence. (2) Exact amount + date within 3 days + M-Pesa transaction ID = 97%. (3) Exact amount + date within 7 days + description cosine similarity >0.85 = 90%. (4) Amount within 2% + date within 7 days + vendor name fuzzy match >0.8 = 80%. (5) Everything else = unmatched, flagged for human review.

Tools: Transaction fetcher (ledger + bank), reference number extractor (regex), M-Pesa ID matcher, description embedder (MiniLM), fuzzy name matcher (rapidfuzz), confidence scorer.

Token Cost: Zero. Self-hosted embedding model + rule engine.

Agent 10 — Payroll Computation Agent: Calculates gross-to-net for every employee. Applies all Kenyan statutory deductions. Generates payslips and statutory return data.

Model: None (pure computation engine). All tax bands, NSSF tiers, NHIF scales, and Housing Levy rates are stored as configuration in the database, not hardcoded. This allows same-day updates when Finance Bill changes occur.

Computation Pipeline: (1) Gross = Basic + House Allowance + Transport + Other. (2) NSSF Employee = 6% of first 7,000 + 6% of (Gross - 7,000) capped at 36,000. (3) Taxable Income = Gross - NSSF Employee. (4) PAYE = progressive bands less personal relief 2,400. (5) NHIF = graduated scale lookup. (6) Housing Levy = 1.5% of Gross. (7) Net = Gross - PAYE - NSSF - NHIF - Housing Levy - Other Deductions.

Tools: Employee data fetcher, tax band calculator, NSSF tier calculator, NHIF scale lookup, payslip PDF generator, P10 CSV exporter, NSSF schedule exporter.

Token Cost: Zero. Pure arithmetic.

Agent 11 — Tax Computation Agent: Calculates VAT liability, WHT liability, instalment tax, and corporate tax estimates. Generates filing-ready CSV/XML files for iTax.

Model: None (pure computation + rule engine). Tax rules are database-configured. VAT3 = Output VAT (from sales) - Input VAT (from purchases). WHT = sum of all WHT withheld on vendor payments. Corporate Tax = estimated from current-period net income.

Tools: Ledger aggregator (sums by tax_category), VAT3 CSV formatter, P10 formatter, WHT return formatter, instalment tax calculator, penalty estimator (what-if: if you file late, this is the penalty).

Token Cost: Zero.

Agent 12 — Inventory Agent: Tracks stock movements. Calculates Weighted Average Cost (WAC). Recognises COGS on sales. Triggers reorder alerts.

Model: None (pure computation). WAC = (existing_qty × existing_WAC + purchase_qty × purchase_unit_cost) / (existing_qty + purchase_qty). Updated perpetually on every purchase.

Tools: Stock ledger (products table), WAC calculator, reorder point checker, inter-branch transfer handler, adjustment logger (with mandatory reason code).

Token Cost: Zero.

TIER 3: VERIFICATION AGENTS (The Auditors)

This is the critical layer that separates Ada Finance from every other accounting tool. Every journal entry proposed by a Tier 2 agent MUST pass through a verification agent before it can be posted. Verification agents are independent — they have no shared state with the proposing agent. They receive only the proposed entry and the source evidence, and they independently assess correctness.

Agent 13 — Double-Entry Verification Agent: Checks that every proposed journal entry satisfies fundamental accounting rules. This is a fast, deterministic check — no LLM needed.

Checks Performed: (1) Total debits = total credits (to the penny). (2) No zero-amount lines. (3) Each line references a valid, active, non-control account. (4) Effective date falls within an open period. (5) No duplicate of an existing entry (same date + amount + vendor within 24 hours). (6) Required evidence is attached. (7) Account types are correct (you can't debit Revenue or credit an Expense in normal operations). (8) Currency is consistent.

Model: None. Pure rule engine. 100% deterministic. <5ms per entry.

Token Cost: Zero.

Failure Action: Rejects the entry with a specific error code. The error is sent back to the proposing agent, which can auto-correct (e.g., fix a rounding error) and resubmit. If auto-correction fails, the entry is flagged for human review via WhatsApp.

Agent 14 — Tax Logic Verification Agent: Independently verifies all tax calculations in a proposed entry. Catches incorrect VAT amounts, wrong WHT rates, miscategorised exempt supplies.

Checks Performed: (1) VAT amount is exactly 16% of the pre-VAT amount (or 0% for exempt/zero-rated items). (2) WHT rate matches the vendor's registered category (Professional 5%, Rent 10%, Contractual 3%, Management/Training 5%, Dividends 5%). (3) If vendor is VAT-registered, Input VAT is claimed; if not, it's expensed. (4) Vendor KRA PIN is valid and active. (5) eTIMS item codes map to correct UNSPSC categories.

Model: Llama 3.2 3B fine-tuned on Kenyan tax rules (self-hosted CPU). Used only for edge cases where rule engine is unsure (e.g., classifying a new type of service for WHT purposes). 90% of checks are pure rule-based.

Token Cost: ~$0.001 per edge case (self-hosted inference). Most entries: $0.

Agent 15 — Semantic Verification Agent (The Second Opinion): This is the most important verification agent. It takes the original source data (the WhatsApp message, the receipt image, the bank statement line) and the proposed journal entry, and independently assesses whether the proposal is a reasonable interpretation of the source.

Why This Exists: Agent 7 (Journal Proposal) might hallucinate an account code that doesn't match the actual transaction. For example, if the receipt says "Carrefour" and the proposal says "DR Telephone Expense", this agent catches it because Carrefour is clearly a retail store, not a telecom.

Model: Claude 3.5 Haiku. Haiku is ideal here because the task is simpler than proposing the entry — it's just answering: "Does this journal entry correctly represent this source document? YES/NO/UNCERTAIN with reason."

Prompt: Minimal prompt ~500 tokens: "Source: [original text/OCR data]. Proposed entry: [journal JSON]. Does this entry correctly represent the source? Answer YES, NO, or UNCERTAIN with a one-sentence reason."

Token Cost: Input: 500 tokens × $0.25/MTok = $0.000125. Output: 50 tokens × $1.25/MTok = $0.0000625. Per verification: $0.000188. At 1,000 entries/month: $0.19/month.

Failure Action: If NO: entry is rejected and sent to human review. If UNCERTAIN: entry is flagged with a warning but can be approved by human. If YES: entry proceeds to execution.

Agent 16 — Anomaly & Fraud Detection Agent: Analyses every proposed entry against historical patterns. Detects duplicates, unusual amounts, suspicious timing, and potential fraud.

Detection Rules: (1) Duplicate detection: same amount + same vendor within 7 days. (2) Unusual amount: >3 standard deviations from the mean for this vendor/category. (3) Round number bias: flags if >80% of a vendor's transactions are suspiciously round (e.g., exactly 10,000, 20,000 — potential fraud indicator). (4) Weekend/holiday transactions from vendors that don't normally transact then. (5) New vendor with high first transaction (>KES 100,000). (6) Expense category suddenly jumps >50% vs. 3-month rolling average.

Model: Isolation Forest (scikit-learn, self-hosted) for statistical anomalies + rule engine for business logic. No LLM needed.

Token Cost: Zero. Pure ML + rules. <10ms per check.

Action: Does NOT block entries. Attaches an anomaly_score and anomaly_reasons array to the entry. If score > threshold, sends WhatsApp alert to Owner. The Owner can dismiss or investigate.

Agent 17 — Reconciliation Verification Agent: After the Bank Reconciliation Agent proposes matches, this agent independently verifies each match. Catches false positives where amount matches but context doesn't.

Model: all-MiniLM-L6-v2 (22M params, self-hosted CPU) for description similarity scoring. Combined with rule checks on date proximity, reference format, and counterparty name.

Token Cost: Zero. Self-hosted.

TIER 4: EXECUTION AGENTS (Write to Ledger, Send to KRA)

Agent 18 — Ledger Posting Agent: The only agent with write access to the journal_entries and ledger_lines tables. No other agent can modify the ledger. This agent receives verified, approved entries and commits them within a PostgreSQL ACID transaction.

Model: None. Pure database transaction handler.

Execution Flow: (1) Begin transaction. (2) Insert journal_entry with status POSTED. (3) Insert all ledger_lines. (4) PostgreSQL trigger validates DR = CR balance. (5) Update running_balance for each affected account. (6) Log in audit_trail. (7) Commit. If any step fails, the entire transaction rolls back.

Security: This agent's database credentials have INSERT-only permissions on journal_entries and ledger_lines. It has no UPDATE or DELETE permissions. Even if compromised, it cannot modify existing records.

Token Cost: Zero.

Agent 19 — eTIMS Submission Agent: Submits invoices and credit notes to KRA's eTIMS VSCU API. Handles retries, offline mode, and error responses.

Model: None. Pure API client with retry logic.

Execution Flow: (1) Validate payload against KRA schema. (2) Submit JSON to VSCU endpoint. (3) If success: stamp invoice with KRA Control Number and QR URL. (4) If failure: save to retry queue with exponential backoff (1min, 5min, 15min, 1hr, 6hr). (5) If KRA is down >6hr: issue invoice with "Pending eTIMS Sync" status. (6) Once KRA is back: flush retry queue.

Token Cost: Zero.

Agent 20 — Payment Disbursement Agent: Initiates M-Pesa B2C payments for supplier payments and salary disbursements. Requires Maker/Checker approval.

Model: None. Pure API client.

Security: Dual-approval required. The agent only executes after both the Preparer AND the Approver (different users) have confirmed. Approvals are logged with timestamp, user ID, and IP address.

Token Cost: Zero.

TIER 5: SYSTEM AGENTS (Self-Improvement, Scheduling, Monitoring)

Agent 21 — Scheduler Agent (The Clock): Manages all time-based triggers in the system. This is the agent that "remembers all accounting cycles."

Implementation: Celery Beat with a dynamic schedule stored in PostgreSQL. Each organisation has its own set of schedules based on its tax obligations and business patterns.

Scheduled Tasks: (1) Email inbox poll: every 60 seconds. (2) M-Pesa reconciliation: every 6 hours. (3) Bank auto-import (where API available): daily at 6am. (4) PAYE alert: 5th of each month. (5) VAT alert: 15th of each month. (6) NSSF alert: 10th of each month. (7) Month-end close reminder: 28th. (8) Recurring transaction proposals: daily at 8am. (9) Anomaly batch scan: daily at midnight. (10) ML model retraining: weekly Sunday 2am. (11) Quarterly instalment tax reminder. (12) Annual P9 generation.

Token Cost: Zero.

Agent 22 — Self-Improvement Agent (The Learner): Continuously improves the accuracy of all ML models in the system. Monitors accuracy metrics. Retrains models. Deploys improvements via shadow testing.

Data Sources: (1) ai_corrections table: every time a user changes an AI-proposed category, account code, or amount. (2) Rejection logs: entries rejected by verification agents. (3) Approval patterns: which types of entries are always approved without changes (high confidence). (4) User feedback: thumbs up/down on WhatsApp responses.

Retraining Pipeline: Weekly: (1) Extract corrections from last 7 days. (2) Add to fine-tuning dataset. (3) Retrain XGBoost categoriser. (4) Update Llama 3.2 3B LoRA adapter for intent classification. (5) Retrain sentence-transformer for description matching. (6) Deploy as shadow model alongside production. (7) Compare accuracy over 48 hours. (8) If shadow is better, promote to production. (9) Log all metrics to Grafana.

Token Cost: ~$5/month for weekly fine-tuning compute on GPU instance.

2. COMPLETE TOKEN COST MODEL

Every model decision in Ada Finance is optimised for the intersection of accuracy, latency, and cost. This section provides a transparent, per-agent cost model for a typical SME processing 1,000 transactions per month.

2.1 Model Selection Matrix

Agent	Model	Input Tokens/Call	Output Tokens/Call	Cost/Call (USD)	Calls/Month	Monthly Cost
WhatsApp Listener	Llama 3.2 3B (CPU)	200	50	$0.000 (self-hosted)	3,000	$0.00
Data Entry (simple)	Claude 3.5 Haiku	500	200	$0.000375	700	$0.26
Data Entry (complex)	Claude 3.5 Sonnet	800	400	$0.0084	300	$2.52
Receipt OCR	PaddleOCR + Donut (GPU)	N/A	N/A	$0.001 (GPU share)	500	$0.50
Email Classifier	Llama 3.2 3B (CPU)	300	30	$0.000 (self-hosted)	2,000	$0.00
Journal Proposal	Claude 3.5 Sonnet	2,000	300	$0.0105	1,000	$10.50
Semantic Verify	Claude 3.5 Haiku	500	50	$0.000188	1,000	$0.19
Tax Logic Verify	Llama 3.2 3B (CPU)	400	100	$0.000 (self-hosted)	200	$0.00
Anomaly Detection	Isolation Forest (CPU)	N/A	N/A	$0.000	1,000	$0.00
Bank Matching	MiniLM-L6 (CPU)	N/A	N/A	$0.000	500	$0.00
All Other Agents	Rule engines / None	N/A	N/A	$0.000	N/A	$0.00
Weekly Retraining	GPU compute	N/A	N/A	$1.25/run	4	$5.00

TOTAL AI/ML COST PER MONTH: $18.97 for an SME processing 1,000 transactions. This is approximately $0.019 per transaction.

2.2 Cost Optimisation Strategies

Strategy 1 — Prompt Caching: Anthropic's prompt caching reduces cost by 90% for repeated system prompts. The Chart of Accounts (~1,500 tokens) and tax rules (~500 tokens) are cached. Only the variable part (the actual transaction) is sent fresh each time. This reduces the Journal Proposal Agent's input cost from $0.006 to $0.0015 per call.

Strategy 2 — Batch API: For non-urgent operations (nightly anomaly scans, monthly report generation), use Anthropic's Batch API which is 50% cheaper than real-time. This saves ~$3/month.

Strategy 3 — Progressive Graduation: As the XGBoost categoriser improves, more transactions can be handled by it (zero cost) instead of Claude. Target: move 50% of Journal Proposal calls to local ML within 6 months, reducing the $10.50/month to $5.25.

Strategy 4 — Fine-tuned Llama for Verification: The Semantic Verification Agent currently uses Haiku. Once we have 10,000+ verified entries, we can fine-tune a Llama 3.2 8B model to replace Haiku for verification, reducing that cost to zero.

2.3 Cost at Scale

Business Size	Transactions/Month	AI/ML Cost/Month	Cost per Transaction
Micro (1-5 employees)	200	$4.50	$0.023
Small (6-20 employees)	1,000	$18.97	$0.019
Medium (21-100 employees)	5,000	$72.00	$0.014
Large (100+ employees)	20,000	$240.00	$0.012

Cost per transaction decreases at scale because fixed costs (GPU instance, self-hosted models) are amortised across more transactions. The marginal cost of each additional transaction is dominated by the Claude API calls ($0.0105 for proposal + $0.000188 for verification = $0.0107 per transaction).

3. AGENT TOOLS & FRAMEWORKS

3.1 Agent Orchestration Framework

Primary Framework: LangGraph (by LangChain). LangGraph is chosen over raw LangChain because it provides stateful, graph-based workflows with explicit control flow. Each agent is a node in the graph. Edges define routing logic. The supervisor node (Orchestrator) decides which agent to invoke based on the current state.

Why Not CrewAI or AutoGen: CrewAI is simpler but lacks fine-grained control over state transitions — critical for accounting where each step must be auditable. AutoGen is conversation-oriented, not task-oriented. LangGraph gives us explicit state machines with full observability.

LangGraph Architecture

```
Orchestrator (Supervisor Node)
  ├── Input Router
  │     ├── WhatsApp Listener Agent → Data Entry Agent
  │     ├── Email Monitor Agent → Bank Parser / OCR Agent
  │     └── M-Pesa Webhook Agent → Receipt Matcher
  ├── Processing Pipeline
  │     ├── Journal Proposal Agent
  │     ├── Payroll Computation Agent
  │     ├── Tax Computation Agent
  │     └── Inventory Agent
  ├── Verification Pipeline (ALL entries pass through here)
  │     ├── Double-Entry Verification Agent
  │     ├── Tax Logic Verification Agent
  │     ├── Semantic Verification Agent
  │     └── Anomaly Detection Agent
  ├── Execution Pipeline
  │     ├── Ledger Posting Agent
  │     ├── eTIMS Submission Agent
  │     └── Payment Disbursement Agent
  └── System Pipeline
        ├── Scheduler Agent
        └── Self-Improvement Agent
```

3.2 Tool Definitions per Agent

Each tool is a Python function registered with the agent via LangChain's @tool decorator. Tools have strict input/output schemas enforced by Pydantic models. An agent cannot call a tool with wrong parameters — the framework rejects it.

Key Tool Libraries

Tool Category	Library/Service	Purpose	Cost

LLM Orchestration	LangGraph + LangChain	Agent workflows, state management, tool calling	Open-source (free)
LLM Inference (cloud)	Anthropic Python SDK	Claude API calls for proposals + verification	$18.97/mo (see above)
LLM Inference (local)	vLLM or Ollama	Serve Llama 3.2 3B/8B models locally	GPU instance cost
OCR	PaddleOCR	Text detection and recognition from images	Open-source (free)
Document AI	Donut (MIT license)	Structured extraction from receipts/invoices	Open-source (free)
Embeddings	sentence-transformers	Description similarity for bank matching	Open-source (free)
ML Classification	scikit-learn + XGBoost	Category prediction, anomaly detection	Open-source (free)
PDF Parsing	tabula-py + pdfplumber	Extract tables from bank statement PDFs	Open-source (free)
Fuzzy Matching	rapidfuzz	Vendor name matching, description similarity	Open-source (free)
Task Queue	Celery + Redis	Async task execution, scheduled jobs	Open-source (free)
PDF Generation	WeasyPrint	Invoice PDFs, payslip PDFs, report PDFs	Open-source (free)
QR Codes	qrcode (Python)	KRA QR codes on invoices	Open-source (free)
Email	imaplib + email (stdlib)	IMAP email reading and attachment extraction	Python stdlib (free)
WhatsApp	Meta Cloud API SDK	Send/receive WhatsApp messages	Per-conversation pricing
M-Pesa	Safaricom Daraja SDK	C2B/B2C transactions	Transaction fees
Observability	LangSmith (LangChain)	Trace every agent call, token usage, latency	Free tier: 5,000 traces/mo

3.3 Agent Communication Protocol

Agents communicate via a standardised message envelope. Every inter-agent message includes:

trace_id: UUID that follows the entire transaction from WhatsApp message to ledger posting. Enables end-to-end tracing in LangSmith.

source_agent: Which agent sent this message.

target_agent: Which agent should receive it.

payload: The actual data (structured JSON with Pydantic validation).

confidence: The sending agent's confidence score (0.0-1.0).

requires_human: Boolean. If true, the Orchestrator pauses and sends a WhatsApp message requesting human approval before continuing.

audit_record: A snapshot of the state at this point, written to the audit_log table.

4. CYBERSECURITY IMPLEMENTATION

Financial data is the highest-value target for attackers. Ada Finance implements defence-in-depth across 7 layers. Every layer is independent — compromising one does not expose the others.

4.1 Security Architecture Overview

Layer	Threat	Control	Implementation
1. Network	DDoS, port scanning, man-in-the-middle	WAF + TLS 1.3 + VPC isolation	AWS WAF with OWASP Top 10 ruleset. CloudFront with TLS 1.3. All services in private VPC subnets (no public IPs except ALB).
2. Authentication	Credential theft, brute force, session hijacking	OTP + JWT + device binding	Phone OTP via Africa's Talking. JWT with 15-minute expiry. Refresh tokens bound to device fingerprint. 5 failed attempts = 30-minute lockout.
3. Authorisation	Privilege escalation, lateral movement	RBAC + Row-Level Security	4 roles (Owner/Manager/Bookkeeper/Auditor). PostgreSQL RLS ensures each query is filtered by org_id. No user can see another org's data even with SQL injection.
4. Application	SQL injection, XSS, CSRF, SSRF	Input validation + ORM + CSP	All inputs validated via Pydantic (FastAPI). SQLAlchemy ORM (no raw SQL). Content-Security-Policy headers. CSRF tokens on all forms.
5. Data	Data breach, exfiltration, tampering	Encryption + WORM + immutability	AES-256 at rest (RDS, S3). TLS 1.3 in transit. S3 Object Lock prevents deletion. Append-only DB tables. KRA PIN and sensitive fields encrypted at application layer (AES-GCM with KMS keys).
6. AI/Agent	Prompt injection, model manipulation, hallucination	Input sanitisation + output validation	All user inputs are sanitised before passing to LLM. Agent outputs are validated against Pydantic schemas. Verification agents catch hallucinated account codes. Agents cannot execute arbitrary code.
7. Monitoring	Undetected breach, insider threat	SIEM + anomaly alerts + audit log	AWS CloudTrail for infrastructure. Application audit_log for every action. Grafana alerts on suspicious patterns (login from new IP, bulk data export, off-hours access).

4.2 Authentication & Registration Flow

User Registration (Key Steps)

Step 1 — Phone Number Entry: User provides their phone number via WhatsApp message ("Hi" to the Ada Finance number) or on the PWA registration page. Phone number is the primary identity.

Step 2 — OTP Verification: 6-digit OTP sent via SMS (Africa's Talking API). OTP expires in 5 minutes. Rate limited: max 5 OTP requests per phone number per hour. Failed attempts logged.

Step 3 — Business Profile Creation: User provides: Business Name, KRA PIN, Industry, Number of Employees. KRA PIN is validated in real-time against iTax API to confirm the business is registered. If PIN is invalid, registration is blocked.

Step 4 — VAT Status Check: System checks if the business is VAT-registered based on KRA PIN validation. If yes, eTIMS integration is enabled. If no, only basic invoicing without eTIMS.

Step 5 — eTIMS Connection (if VAT-registered): User provides eTIMS Communication Key (obtained from KRA). System registers as a VSCU device in KRA sandbox. Performs a test invoice submission to verify connectivity.

Step 6 — M-Pesa Linking: User provides Paybill/Till number. System registers C2B webhook via Daraja API. Performs a test transaction of KES 1 to verify the link. User confirms receipt.

Step 7 — Bank Account Setup: User selects their primary bank. For CSV import banks, uploads a sample statement. System auto-detects the bank format. For email-forwarded statements, provides a dedicated email address.

Step 8 — Team Invitation: Owner invites Bookkeeper, Manager, External Accountant via phone number. Each invitee receives a WhatsApp message with a registration link. Roles are assigned at invitation.

Step 9 — Chart of Accounts: System auto-generates a standard Kenyan SME Chart of Accounts based on the selected industry. Owner can customise (add/rename accounts). If migrating from another system, upload opening Trial Balance.

Step 10 — First Transaction: Guided walkthrough via WhatsApp. System asks: “Record your first sale! Type something like: Sold 10 bags of cement to ABC Hardware for 15,000 via M-Pesa.” User types a transaction. AI proposes entry. User approves. Onboarding complete.

Authentication Tokens

Access Token: JWT. 15-minute expiry. Contains: user_id, org_id, role, permissions[]. Signed with RS256 (asymmetric key pair). The short expiry limits exposure if token is intercepted.

Refresh Token: Opaque token stored in HttpOnly, Secure, SameSite=Strict cookie. 30-day expiry. Bound to device fingerprint (user-agent + screen resolution hash). If device changes, refresh token is invalidated.

WhatsApp Session: Phone number is the session key. Validated via initial OTP. All subsequent WhatsApp messages from this number are authenticated. Session expires after 30 days of inactivity. Re-verification required for sensitive operations (payment approval, period close).

Auditor Token: Time-limited (7-day) read-only token. Generated by Owner. Grants access only to specified date ranges. Cannot be refreshed — auditor must request a new token.

4.3 Sensitive Data Handling

KRA PINs: Encrypted at application layer using AES-256-GCM with AWS KMS-managed keys. Stored as encrypted blobs in PostgreSQL. Decrypted only when needed for API calls to KRA. Never logged in plaintext.

M-Pesa Transaction Data: Phone numbers are hashed (SHA-256 with salt) in analytics tables. Full phone numbers exist only in the operational tables, encrypted at rest via RDS encryption.

Payroll Data: Salary information is classified as HIGHLY SENSITIVE. Accessible only to Owner role and the Payroll Computation Agent. Bookkeeper role cannot view individual salaries unless explicitly granted access by Owner.

Bank Statements: Stored in S3 with server-side encryption (SSE-S3). Access logged via S3 access logs. Retention: 7 years with Object Lock.

Audit Log: Append-only. The audit_log table has no UPDATE or DELETE triggers. The PostgreSQL user that writes audit records has INSERT-only permissions. Even database administrators cannot delete audit records without rebuilding the table (which would be detected by checksum verification).

4.4 Cybersecurity Tooling

Function	Tool	Type	Cost
Web Application Firewall	AWS WAF	Managed	$5/month + $0.60/million requests
DDoS Protection	AWS Shield Standard	Managed	Free (included with ALB)
TLS Termination	AWS CloudFront + ACM	Managed	Free certificates, CloudFront pricing

Secret Management	AWS Secrets Manager	Managed	$0.40/secret/month
Encryption Key Management	AWS KMS	Managed	$1/key/month + $0.03/10,000 requests
Vulnerability Scanning	Snyk (open-source tier)	SaaS	Free for open-source dependencies
Container Scanning	Trivy	Open-source	Free
SAST (Static Analysis)	Bandit (Python) + ESLint	Open-source	Free
Dependency Audit	pip-audit + npm audit	Open-source	Free
Runtime Protection	AWS GuardDuty	Managed	~$10/month for small workloads
Log Aggregation	AWS CloudWatch Logs	Managed	$0.50/GB ingested
Error Tracking	Sentry	SaaS	Free tier: 5,000 errors/month
Penetration Testing	Manual (quarterly)	Consultant	$2,000-5,000/quarter
Compliance	SOC 2 Type I (target Year 2)	Audit	$10,000-20,000 one-time

TOTAL CYBERSECURITY COST: ~$25-40/month for infrastructure security tooling (excluding penetration testing and SOC 2 audit). This is built into the overall infrastructure budget.

5. BACKEND ARCHITECTURE DESIGN

5.1 API Layer Design (FastAPI)

The backend is built on Python FastAPI, chosen for its async support (critical for handling WhatsApp webhooks and concurrent agent tasks), automatic OpenAPI documentation, Pydantic model validation, and performance (comparable to Node.js/Go for I/O-bound workloads).

Service Architecture

The backend follows a modular monolith pattern (not microservices). All services run in the same container but are logically separated into modules with clean interfaces. This gives the speed of a monolith (no network hops between services) with the maintainability of microservices (each module can be extracted later if needed).

```
ada-finance-api/
  ├── app/
  │    ├── main.py                  # FastAPI app, middleware, CORS
  │    ├── config.py                # Environment variables, secrets
  │    ├── database.py              # SQLAlchemy engine, session factory
  │    ├── auth/
  │    │    ├── router.py            # /auth/register, /auth/otp, /auth/refresh
  │    │    ├── service.py           # OTP generation, JWT issuance, RBAC
  │    │    └── models.py            # User, Organisation, Role schemas
  │    ├── ledger/
  │    │    ├── router.py            # /journals, /ledger, /trial-balance
  │    │    ├── service.py           # Journal creation, posting, reversal
  │    │    └── models.py            # JournalEntry, LedgerLine, Account
  │    ├── sales/
  │    │    ├── router.py            # /invoices, /receipts, /credit-notes
  │    │    └── service.py           # Invoice generation, eTIMS, AR
  │    ├── purchases/
  │    │    ├── router.py            # /bills, /payments, /wht
  │    │    └── service.py           # Bill processing, WHT, AP
  │    ├── banking/
  │    │    ├── router.py            # /bank-import, /reconciliation, /mpesa
  │    │    └── service.py           # Statement parsing, matching, Daraja
  │    ├── payroll/
  │    │    ├── router.py            # /payroll/run, /payroll/payslips
  │    │    └── service.py           # Gross-to-net, statutory deductions
  │    ├── tax/
  │    │    ├── router.py            # /tax/vat, /tax/paye, /tax/dashboard
  │    │    └── service.py           # Tax calculations, return generation
  │    ├── inventory/
  │    │    ├── router.py            # /products, /stock, /adjustments
  │    │    └── service.py           # WAC, stock movements, reorders
  │    ├── reports/
  │    │    ├── router.py            # /reports/pnl, /reports/bs, /reports/cashflow
  │    │    └── service.py           # Financial statement generation
```

```
│     ├── whatsapp/
│     │     ├── router.py            # /webhook/whatsapp (Meta callback)
│     │     └── service.py           # Message handling, media download
│     ├── agents/
│     │     ├── orchestrator.py      # LangGraph supervisor
│     │     ├── input_agents.py      # Agents 1-6
│     │     ├── processing_agents.py # Agents 7-12
│     │     ├── verify_agents.py     # Agents 13-17
│     │     ├── exec_agents.py       # Agents 18-20
│     │     └── system_agents.py     # Agents 21-22
│     └── audit/
│           ├── router.py            # /audit/log, /audit/portal
│           └── service.py           # Audit log queries, auditor access
├── workers/
│     ├── celery_app.py            # Celery configuration
│     ├── tasks/
│     │     ├── email_tasks.py       # Email polling, classification
│     │     ├── ocr_tasks.py         # Receipt/invoice processing
│     │     ├── etims_tasks.py       # KRA submission + retry
│     │     ├── recon_tasks.py       # Bank reconciliation
│     │     ├── payroll_tasks.py     # Payroll batch processing
│     │     └── ml_tasks.py          # Model retraining
│     └── beat_schedule.py         # Celery Beat periodic tasks
├── alembic/                          # Database migrations
├── tests/                            # pytest test suite
├── Dockerfile
├── docker-compose.yml
└── requirements.txt
```

5.2 Database Design Patterns

Multi-Tenancy

Ada Finance uses a shared-database, shared-schema multi-tenancy model with PostgreSQL Row-Level Security (RLS). Every table has an org_id column. RLS policies ensure that every SELECT, INSERT, UPDATE, and DELETE is automatically filtered by the authenticated user's org_id. Even if a developer writes a query without a WHERE org_id = clause, RLS adds it automatically. This is the strongest form of data isolation without separate databases.

```
-- Enable RLS on all tables
ALTER TABLE journal_entries ENABLE ROW LEVEL SECURITY;
CREATE POLICY org_isolation ON journal_entries
  USING (org_id = current_setting('app.current_org_id')::uuid);
-- Set org_id on every API request via FastAPI middleware
SET app.current_org_id = '{user_org_id}';
```

Append-Only Enforcement

The journal_entries and ledger_lines tables use PostgreSQL triggers to block UPDATE and DELETE operations. The trigger raises an exception on any non-INSERT operation. Even a database superuser cannot modify these records without disabling the trigger (which is logged by AWS CloudTrail).

```
CREATE OR REPLACE FUNCTION block_modification()
RETURNS TRIGGER AS $$
BEGIN
  RAISE EXCEPTION 'Modification of financial records is prohibited.';
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER no_update_journal BEFORE UPDATE ON journal_entries
  FOR EACH ROW EXECUTE FUNCTION block_modification();
CREATE TRIGGER no_delete_journal BEFORE DELETE ON journal_entries
  FOR EACH ROW EXECUTE FUNCTION block_modification();
```

API Endpoint Summary

Module	Endpoint	Method	Auth	Description
Auth	/auth/register	POST	Public	Register new organisation + owner
Auth	/auth/otp/request	POST	Public	Request OTP for phone number
Auth	/auth/otp/verify	POST	Public	Verify OTP, receive JWT
Auth	/auth/refresh	POST	Cookie	Refresh access token
Auth	/auth/invite	POST	Owner	Invite team member
Ledger	/journals	POST	Bookkeeper+	Create draft journal entry
Ledger	/journals/{id}/approve	POST	Manager+	Approve and post entry
Ledger	/journals/{id}/reverse	POST	Manager+	Reverse a posted entry
Ledger	/trial-balance	GET	Bookkeeper+	Get current trial balance
Ledger	/ledger/{account}	GET	Bookkeeper+	Get account transactions
Sales	/invoices	POST	Bookkeeper+	Create sales invoice
Sales	/invoices/{id}/send	POST	Bookkeeper+	Send invoice via WhatsApp
Purchases	/bills	POST	Bookkeeper+	Create supplier bill
Purchases	/payments	POST	Manager+	Create payment (requires approval)
Banking	/bank/import	POST	Bookkeeper+	Upload bank statement
Banking	/bank/reconcile	POST	Bookkeeper+	Run auto-reconciliation
Payroll	/payroll/run	POST	Manager+	Execute payroll for period
Tax	/tax/vat3	GET	Bookkeeper+	Generate VAT3 return data
Tax	/tax/dashboard	GET	Owner+	Compliance status overview
Reports	/reports/pnl	GET	Bookkeeper+	Profit & Loss statement
Reports	/reports/balance-sheet	GET	Bookkeeper+	Balance Sheet
WhatsApp	/webhook/whatsapp	POST	Meta API	Incoming WhatsApp messages
M-Pesa	/webhook/mpesa/c2b	POST	Daraja	Incoming M-Pesa payments
Audit	/audit/log	GET	Owner/Auditor	Query audit trail

5.3 Async Processing Architecture

All heavy processing (OCR, AI journal proposals, bank reconciliation, eTIMS submissions) runs asynchronously via Celery workers. The API layer never blocks — it enqueues tasks and returns immediately. Results are delivered via WhatsApp message or WebSocket push to the PWA.

Queue Priority: Three priority queues: (1) HIGH: WhatsApp responses, payment approvals (must respond within 5 seconds). (2) MEDIUM: journal proposals, OCR extraction, eTIMS submissions (within 30 seconds). (3) LOW: reports, retraining, batch reconciliation (within 5 minutes).

Worker Scaling: Celery workers auto-scale based on queue depth. Minimum 2 workers always running. Max 10 workers during peak hours (month-end). GPU workers are dedicated and do not auto-scale (single g4dn.xlarge instance handles all OCR/ML inference).

6. THE LEDGER: COMPLETE BUILD GUIDE

6.1 Why the Ledger Must Be In-App

The ledger is the single source of truth. It cannot be delegated to QuickBooks, Xero, or any external system. Reasons: (1) Zero-latency reads for real-time financial statements. (2) Full control over Kenyan tax logic (eTIMS, WHT, Housing Levy). (3) Append-only immutability enforced at database level. (4) No third-party API rate limits. (5) No licensing costs. (6) Custom multi-tenant RLS. (7) 7-year retention with WORM compliance. QuickBooks cannot provide points 3, 6, or 7.

6.2 Complete DDL (SQL Schema)

The following is the complete PostgreSQL DDL for the Ada Finance ledger. This script creates all core tables, triggers, constraints, indexes, and functions. It is idempotent (safe to run multiple times).

Core Ledger Tables

```
-- 1. ACCOUNTS (Chart of Accounts)
CREATE TABLE accounts (
  code        VARCHAR(10) PRIMARY KEY,
  org_id      UUID NOT NULL REFERENCES organisations(id),
  name        VARCHAR(100) NOT NULL,
  type        VARCHAR(20) NOT NULL CHECK (type IN
('ASSET','LIABILITY','EQUITY','REVENUE','EXPENSE')),
  normal_bal  VARCHAR(2) NOT NULL CHECK (normal_bal IN ('DR','CR')),
  is_control  BOOLEAN DEFAULT false,
  is_active   BOOLEAN DEFAULT true,
  parent_code VARCHAR(10) REFERENCES accounts(code),
  tax_cat     VARCHAR(30) DEFAULT 'NONE',
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- 2. PERIODS (Accounting periods with state machine)
CREATE TABLE periods (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL REFERENCES organisations(id),
  start_date  DATE NOT NULL,
  end_date    DATE NOT NULL,
  status      VARCHAR(15) DEFAULT 'OPEN'
              CHECK (status IN ('OPEN','SOFT_CLOSED','HARD_CLOSED','LOCKED')),
  closed_by   UUID REFERENCES users(id),
  closed_at   TIMESTAMPTZ
);

-- 3. JOURNAL ENTRIES (Master transaction record - APPEND ONLY)
CREATE TABLE journal_entries (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id          UUID NOT NULL REFERENCES organisations(id),
  effective_date  DATE NOT NULL,
```

```
  period_id       UUID NOT NULL REFERENCES periods(id),
  created_by      UUID NOT NULL REFERENCES users(id),
  approved_by     UUID REFERENCES users(id),
  status          VARCHAR(20) DEFAULT 'DRAFT'
                  CHECK (status IN ('DRAFT','PENDING','POSTED','REVERSED')),
  description     TEXT NOT NULL,
  source_type     VARCHAR(30),
  source_ref      VARCHAR(200),
  reversed_by     UUID REFERENCES journal_entries(id),
  reversal_reason TEXT,
  evidence_id     UUID,
  ai_confidence   DECIMAL(5,4),
  ai_model        VARCHAR(50),
  metadata        JSONB DEFAULT '{}',
  created_at      TIMESTAMPTZ DEFAULT now()
);

-- 4. LEDGER LINES (Individual debit/credit lines)
CREATE TABLE ledger_lines (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  journal_entry_id UUID NOT NULL REFERENCES journal_entries(id),
  account_code     VARCHAR(10) NOT NULL REFERENCES accounts(code),
  debit            DECIMAL(15,2) NOT NULL DEFAULT 0.00,
  credit           DECIMAL(15,2) NOT NULL DEFAULT 0.00,
  running_balance  DECIMAL(15,2),
  description      TEXT,
  CHECK (debit >= 0 AND credit >= 0),
  CHECK (debit = 0 OR credit = 0)
);
```

Critical Triggers

```
-- TRIGGER: Block UPDATE/DELETE on journal_entries
CREATE OR REPLACE FUNCTION block_modification() RETURNS TRIGGER AS $$
BEGIN RAISE EXCEPTION 'Modification of financial records is prohibited'; END; $$ LANGUAGE
plpgsql;
CREATE TRIGGER no_update_je BEFORE UPDATE ON journal_entries FOR EACH ROW EXECUTE FUNCTION
block_modification();
CREATE TRIGGER no_delete_je BEFORE DELETE ON journal_entries FOR EACH ROW EXECUTE FUNCTION
block_modification();
CREATE TRIGGER no_update_ll BEFORE UPDATE ON ledger_lines FOR EACH ROW EXECUTE FUNCTION
block_modification();
CREATE TRIGGER no_delete_ll BEFORE DELETE ON ledger_lines FOR EACH ROW EXECUTE FUNCTION
block_modification();

-- TRIGGER: Enforce balanced entries (DR = CR)
CREATE OR REPLACE FUNCTION enforce_balance() RETURNS TRIGGER AS $$
BEGIN
  IF (SELECT ABS(SUM(debit) - SUM(credit)) FROM ledger_lines
```

```
      WHERE journal_entry_id = NEW.journal_entry_id) > 0.005 THEN
    RAISE EXCEPTION 'Entry does not balance: total DR <> total CR';
  END IF; RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE CONSTRAINT TRIGGER check_balance AFTER INSERT ON ledger_lines
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION enforce_balance();

-- TRIGGER: Update running balance
CREATE OR REPLACE FUNCTION update_running_balance() RETURNS TRIGGER AS $$
DECLARE prev_balance DECIMAL(15,2);
BEGIN
  SELECT COALESCE(running_balance, 0) INTO prev_balance FROM ledger_lines
    WHERE account_code = NEW.account_code ORDER BY created_at DESC LIMIT 1;
  NEW.running_balance := prev_balance + NEW.debit - NEW.credit;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER calc_running_bal BEFORE INSERT ON ledger_lines
  FOR EACH ROW EXECUTE FUNCTION update_running_balance();
```

Performance Indexes

```
CREATE INDEX idx_ll_account ON ledger_lines(account_code, created_at DESC);
CREATE INDEX idx_ll_journal ON ledger_lines(journal_entry_id);
CREATE INDEX idx_je_period ON journal_entries(period_id, status);
CREATE INDEX idx_je_org_date ON journal_entries(org_id, effective_date DESC);
CREATE INDEX idx_je_source ON journal_entries(source_type, source_ref);
```

6.3 How the Ledger Fills Itself

The ledger is populated by the Ledger Posting Agent (Agent 18), which is the ONLY component with INSERT access to journal_entries and ledger_lines. Here is the complete flow from WhatsApp message to posted ledger entry:

Step 1: User sends “Paid 5k for fuel via M-Pesa” to WhatsApp.

Step 2: WhatsApp Listener Agent (Agent 1) classifies intent as TRANSACTION_RECORDING.

Step 3: Data Entry Agent (Agent 2, Claude Haiku) extracts: {type: expense, amount: 5000, category: fuel, method: mpesa}.

Step 4: Journal Proposal Agent (Agent 7, Claude Sonnet) proposes: DR 5030 Fuel Expense 5,000 / CR 1015 M-Pesa 5,000.

Step 5: Double-Entry Verification Agent (Agent 13) checks: DR = CR? Yes. Accounts valid? Yes. Period open? Yes.

Step 6: Tax Logic Verification Agent (Agent 14) checks: VAT applicable on fuel? Yes (but no separate VAT line for cash purchase). WHT? No.

Step 7: Semantic Verification Agent (Agent 15, Claude Haiku) checks: Does “fuel” match “Fuel Expense” account? Yes. Does “M-Pesa” match “M-Pesa” account? Yes. PASS.

Step 8: Anomaly Detection Agent (Agent 16) checks: Amount normal for fuel? Yes. No duplicates. PASS.

Step 9: Orchestrator sends WhatsApp message to user: “Drafted: DR Fuel Expense 5,000 / CR M-Pesa 5,000. Reply YES to post.”

Step 10: User replies YES.

Step 11: Ledger Posting Agent (Agent 18) executes PostgreSQL transaction: INSERT journal_entry, INSERT 2 ledger_lines, triggers validate, COMMIT.

Step 12: Confirmation sent via WhatsApp: “Posted. Balance: Fuel Expense 23,500 | M-Pesa 142,300.”

Total time from Step 1 to Step 12: 3-8 seconds (depending on user approval speed). AI processing time: <2 seconds. The bottleneck is waiting for the user to reply YES.

7. MODULE-BY-MODULE EXECUTION (SPEED-OPTIMISED)

Every module is designed for maximum speed. We use parallel tracks where dependencies allow. Total calendar time: 20 working days with 3 engineers. Compressible to 15 days with 4 engineers.

Day	Track A (Lead Engineer)	Track B (Backend Engineer)	Track C (AI Engineer)	Validation
0	AWS infra, CI/CD, Docker	PostgreSQL DDL, seed COA	GPU instance, deploy OCR models	Infra health check
1	Auth: registration, OTP, JWT	Ledger API: journal CRUD	—	—
2	Auth: RBAC, RLS policies	Ledger: approval, reversal, TB	—	GATE 1: Post entries, TB balances
3	PWA: registration flow	WhatsApp webhook handler	NLP intent classifier (Llama)	—
4	—	WhatsApp → Journal flow	Receipt OCR pipeline	GATE 2: WhatsApp entries work
5	Invoice PDF + QR generation	eTIMS sandbox integration	Data Entry Agent (Claude)	—
6	Customer receipts + AR	eTIMS retry queue	—	GATE 3: eTIMS invoice + QR
7	Supplier bills API	WHT auto-calculation	OCR for supplier invoices	—
8	Payment approval flow	Bank CSV parser (6 banks)	—	GATE 4: Bill + WHT + approval
9	M-Pesa Daraja C2B/B2C	Bank reconciliation engine	MiniLM fuzzy matcher	—
10	Reconciliation report + UI	Cash position dashboard	—	GATE 5: 80%+ auto-match
11	Payroll engine (PAYE/NSSF)	Payslip PDF + P10 export	Email Monitor Agent	—
12	Payroll approval + M-Pesa B2C	—	Email classification + routing	GATE 6: Payroll matches KRA
13	Tax computation engine	VAT3 + WHT return gen	Verification agents (13-17)	—
14	Compliance dashboard	—	Anomaly detection agent	GATE 7: Tax returns correct
15	P&L + Balance Sheet	Cash Flow statement	Self-improvement agent	—
16	Management dashboards	Auditor portal (read-only)	—	GATE 8: Financials balance
17	Inventory: stock + COGS + WAC	Audit trail enforcement	End-to-end agent testing	—
18	Month-end controls UI	Period state machine	—	GATE 9: Full audit trail
19	Integration testing	Load testing	Agent accuracy testing	GATE 10: Full system test
20	Production deploy	Data migration tools	Monitoring + alerting	GO-LIVE

8. COMPLETE COST SUMMARY

Category	Item	Monthly Cost (USD)
Compute	ECS Fargate (API + Workers)	$120
Compute	GPU Instance (g4dn.xlarge spot)	$95
Database	RDS PostgreSQL (Multi-AZ)	$250
Cache	ElastiCache Redis	$65
Storage	S3 + Glacier (7-year)	$15
AI API	Anthropic Claude (Sonnet + Haiku)	$19
Messaging	Meta WhatsApp Cloud API	$80
SMS	Africa's Talking (OTP + alerts)	$30
Security	WAF + KMS + GuardDuty	$35
Monitoring	CloudWatch + Sentry	$15
Domain	adafinance.co.ke + SSL	$5
	TOTAL INFRASTRUCTURE	$729/month
	TOTAL PER SME CLIENT (at 50 clients)	$14.58/month

At 50 SME clients, the infrastructure cost per client is under $15/month. With a subscription price of $30-50/month (KES 4,000-6,500), the gross margin is 50-70%. The AI/ML cost is the most scalable component — it grows linearly with transactions but decreases per-unit with scale.

9. TECHNOLOGY STACK FINAL SUMMARY

Layer	Choice	Why
Backend Framework	Python FastAPI + Celery	Async, fast, Pydantic validation, LangChain native
Frontend	Next.js 14 + Tailwind + shadcn/ui	SSR, PWA, rapid UI, offline capable
Mobile	React Native (Expo)	Code sharing with web, OTA updates
Database	PostgreSQL 16 + TimescaleDB	ACID, append-only, RLS, time-series analytics
Cache/Queue	Redis 7 + Celery	Sub-ms cache, task queue, pub/sub
Object Storage	AWS S3 (Object Lock)	WORM compliance, 7-year retention
Agent Framework	LangGraph + LangChain	Stateful agent workflows, tool calling, tracing
LLM (Complex)	Anthropic Claude 3.5 Sonnet	Best accuracy for financial structured output
LLM (Simple)	Anthropic Claude 3.5 Haiku	80% cheaper, good for verification
LLM (Local)	Llama 3.2 3B (vLLM)	Zero API cost, intent classification
OCR	PaddleOCR + Donut	Zero API cost, self-hosted, 95%+ accuracy
ML	XGBoost + Isolation Forest	Zero cost, sub-ms inference, deterministic
Embeddings	all-MiniLM-L6-v2	22M params, CPU, fuzzy description matching
WhatsApp	Meta Cloud API (direct)	No Twilio markup, lower latency
M-Pesa	Safaricom Daraja API	Official, C2B + B2C
KRA	eTIMS VSCU REST API	Direct integration
Auth	Custom (JWT + OTP)	Phone-first, no third-party dependency
Infrastructure	AWS (af-south-1) + ECS Fargate	Cape Town region, serverless containers

CI/CD	GitHub Actions + Docker + ECR	Automated build, test, deploy
Monitoring	Grafana + Prometheus + Sentry	Metrics, alerting, error tracking
Agent Tracing	LangSmith	Token usage, latency, accuracy per agent

10. IMMEDIATE NEXT STEPS

TODAY: Sign off on this implementation plan. Confirm the 20-day timeline. Assign the 3-engineer team.

DAY 0 — Access & Credentials: Obtain KRA eTIMS sandbox credentials. Register Meta WhatsApp Business API. Register Safaricom Daraja sandbox. Provision AWS account in af-south-1. Register adafinance.co.ke domain.

DAY 0 — Database: Run the complete DDL script. Seed Kenyan standard Chart of Accounts. Seed tax calendar (PAYE 9th, VAT 20th, NSSF 15th deadlines). Seed 2026 PAYE tax bands.

DAY 0 — CI/CD: Create GitHub repository with branch protection. Configure GitHub Actions pipeline. Build base Docker images for API and workers.

DAY 1 — Sprint Kickoff: Begin Module 0 (Infrastructure) and Module 1 (Core Ledger) in parallel. Daily standup at 9am EAT. Friday validation gate reviews.

ADA FINANCE

22 Agents. 4 Verification Layers. 20 Days to Go-Live.

Zero-effort compliance. Built for Kenya. Powered by Intelligence.